Node.js 运行时升级报告

Oct 22, 2026

结论：AI 助手服务（ai_assistant）线上跑的是 Node 18，已于 2025-04-30 停止维护。 2026-10-21 公布的 Node.js 高危漏洞它都在影响范围内，而且官方不会再出补丁，只能升级到 22.23.4。前端构建用的 Node 21 也已停止维护，建议一并升级。

问题是什么

2026-10-21，Node.js 官方发布安全更新，一次修复 10 个漏洞，其中 4 个是高危。修复版本是 22.23.4、24.22.1 和 26.12.1。

漏洞编号	问题	可能的后果	受影响的维护中版本
CVE-2026-111497	Brotli 解压缩时越界写	程序解压攻击者构造的 Brotli 数据时，内存可能泄露，攻击者也可能远程执行任意代码	22 / 24 / 26
CVE-2026-106069	TLS 证书主机名校验可被绕过（主机名能被宽松解析成 IP 时）	HTTPS 客户端可能信任冒充的服务器	22 / 24 / 26
CVE-2026-105483	TLS 证书主机名校验可被绕过（IP 用全角数字书写时）	同上	24 / 26
CVE-2026-105482	`dns.resolve` 遇到含 NUL 字符的域名会截断	可能解析到非预期的域名	22 / 24 / 26

另外还有 3 个中危和 3 个低危漏洞，内置的 OpenSSL、c-ares、undici 也随版本一起更新了。官方公告还说明：每次发布安全更新，已停止维护的版本一律视为受影响，而且不会修复。

为什么只能靠升级解决

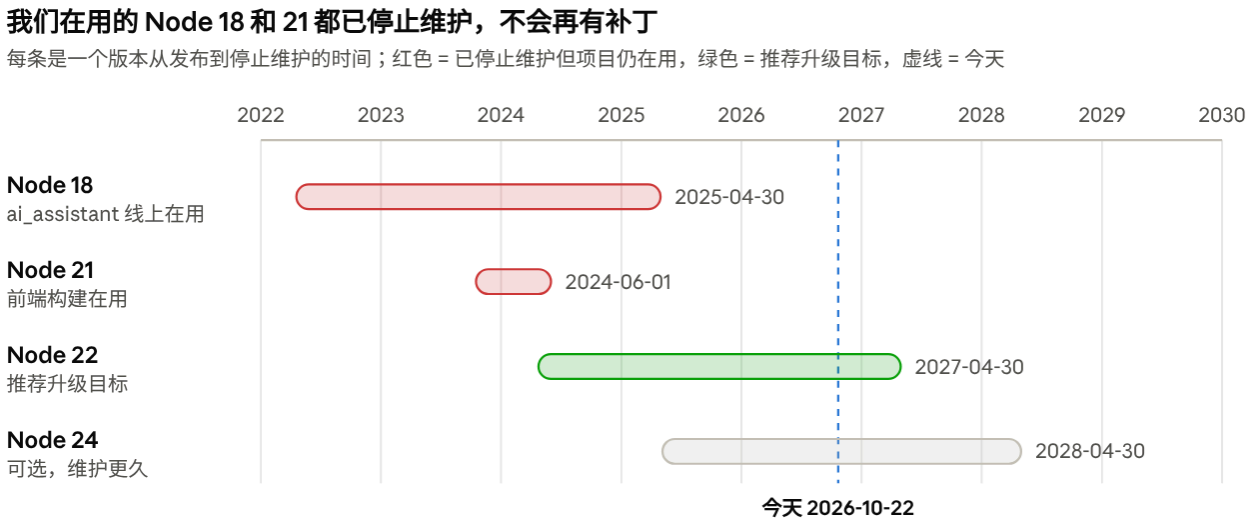
根本原因是我们用的 Node 版本已经没人维护了。这次的漏洞只是把问题暴露了出来。

- **漏洞在 Node 本体里，不在我们的代码里。** 出问题的是 Node 自带的 Brotli、TLS、DNS 模块。改业务代码、升级 npm 包都绕不开，官方也没有给出临时规避办法，只能换成修复过的 Node 版本。
- **Node 18 已于 2025-04-30 停止维护。** ai_assistant 线上跑的就是它。到今天已经停了约 18 个月，这期间 Node 的每一次安全更新都没有给它修。

- **Node 21 已于 2024-06-01 停止维护。** 前端构建用的就是它。它是奇数版，从来不是长期支持版（LTS），发布后只维护了约 7 个月。
- **升级是把版本换回有人维护的线。** 换到 22.23.4，这次的 10 个漏洞一次修掉。之后的安全更新也能拿到，直到 Node 22 在 2027-04-30 停止维护。

推荐 22.23.4：从 18 和 21 过去跨度最小。想要更长的维护期可以选 24.22.1（维护到 2028-04-30），但升级跨度更大，需要多做一些回归测试。

版本支持周期



Node.js 各版本维护期 · 4 个版本

Node 18 已停止维护约 18 个月，Node 21 已停止超过 2 年。换到 Node 22 后，可以一直拿到安全补丁直到 2027-04-30。

影响范围

项目里有 3 处用到 Node，全部低于修复版本。其中只有 ai_assistant 在线上实际运行 Node。

位置	当前版本	用途	线上是否运行 Node	结论
ai_assistant（后端 AI 助手服务）：`CSS_backend-main/deploy/compose/backend.yml` 里的内联镜像，另外还有项目自带的 `ai_assistant/Dockerfile`	Node 18（内联镜像 `node:18-bookworm-slim`，自带 Dockerfile 是 `node:18-alpine`）	随后端服务部署到 test / uat / prod，监听 2025 端口	是	受影响，而且永远不会有补丁，**必须升级**
前端构建：`CSS_frontend-main/Dockerfile`、`Dockerfile.novalidate`、前端 CI `cicd.yml`	Node 21（`node:21-alpine`，CI 里是 `node-version: 21`）	只在构建阶段用，最终镜像是 nginx	否	受影响但风险低，建议一并升级
开发本机（nvm）	Node 22.14.0	本地跑前端和 ai_assistant	否	受影响，升到 22.23.4

后端主 API、OCR、短信和邮件发送服务都是 Python 写的，不受这次漏洞影响。

风险评估

眼下被直接利用的可能性较低，但长期风险高。拖得越久，累积的未修漏洞就越多。

为什么眼下风险较低

- **Brotli 漏洞**：只有程序去解压攻击者控制的 Brotli 数据时才会被触发。
  - 收请求：ai_assistant 用 Express 4 接收请求，这一版本不会解压 Brotli 格式的请求体。
  - 发请求：它用 node-fetch 2.7.0 对外发请求。核对源码后确认，对方响应头带 `Content-Encoding: br` 时它会自动解压。不过它只连两个地方：OpenAI（走 HTTPS）和内网的主 API `http://app:8000`，都是可信的服务。
- **TLS 两个漏洞**：从标题看，和“把 IP 地址当主机名”的情况有关，而 ai_assistant 连的是域名 api.openai.com。公告没有说只限于 IP 的情况，所以不能完全排除。
- **暴露面**：前端和后端的 nginx 都没有把任何路径转发到 2025 端口。应用机在私有子网里，所以从代码看，公网访问不到 ai_assistant。安全组配置暂未核实。

为什么长期风险高

- Node 18 以后的每一次安全更新都不会给它修。下一个漏洞如果是入站方向的，就没有底可兜了。
- 内网的信任不是永久的。内网里只要有一台服务被攻破，攻击者就能向 ai_assistant 返回恶意的 Brotli 响应，把攻击扩散到它所在的应用机。这台应用机上还跑着主 API。
- 安全扫描和客户审计会把“停止维护的运行时”单独列为风险项。

升级方案

改动只涉及镜像和 CI 里的版本号，业务代码不用改。版本号一律写死为 `22.23.4`，不要用 `node:22` 这类浮动标签，确保拉到的一定是修复后的版本。

1. **ai_assistant**：`deploy/compose/backend.yml` 里的内联镜像从 `node:18-bookworm-slim` 改为 `node:22.23.4-bookworm-slim`；`ai_assistant/Dockerfile` 里的 `node:18-alpine` 同步改为 `node:22.23.4-alpine`。它依赖的 canvas 3.1.0 在 Node 22 上可以正常使用。
2. **前端**：两个 Dockerfile 里的 `node:21-alpine` 改为 `node:22.23.4-alpine`；前端 CI 里 3 处 `node-version: 21` 改为 `22.23.4`。
3. **本机**：先执行 `nvm install 22.23.4`，再执行 `nvm use 22.23.4`。
4. **发布**：按 ops → test、ops → uat、ops → main 的路径晋级。test 和 uat push 后自动部署。prod 要在 GitHub Actions 里手动 Run workflow（environment=prod，image_tag 留空）。三个 support 环境的应用机重新开机前也要重新部署。
5. **回归测试**：在 test 环境把 AI 助手对话和生成图表走一遍，没问题再发 uat 和 prod。
6. **确认生效**：在每台应用机上对 ai_assistant 容器执行 `node -v`，应输出 `v22.23.4`。还可以用第三方 Atlas 给出的检查方法：执行 `node -p "require('zlib').constants.BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH || 'missing'"`，已修复的版本输出 `-17`。本机 22.14.0 实测输出的是 `missing`，说明还没修。

这次升级不处理 npm 依赖包和基础镜像系统包里的漏洞，这两类需要另外用 `npm audit` 和镜像扫描来检查。

来源

- Node.js 官方安全公告：2026-10-21 Security Releases
- Node.js v22.23.4 发布说明
- Node.js 发布与维护计划（nodejs/Release）
- Vercel：Node.js security advisory: Brotli decoder RCE
- Atlas：CVE-2026-111497 Node.js Brotli Decoder 分析（第三方博客，补丁检查方法和 Express 4 的行为都出自这里）

- node-fetch 2.7.0 源码（自动解压 br 响应那一段）
- 项目内核对：`CSS_backend-main/deploy/compose/backend.yml`、`CSS_backend-main/ai_assistant/`、`CSS_frontend-main/Dockerfile*`、`CSS_frontend-main/.github/workflows/cicd.yml`、前后端 nginx 配置